```
`timescale 1ns / 1ps
`default_nettype none

module reg_file_tb;

    // Inputs
    reg clk;
    reg rst_n;
    reg i_re;
    reg i_wr;
    reg i_amo_en;
    reg [4:0] i_rs1;
    reg [4:0] i_rs2;
    reg [4:0] i_rd;
    reg signed [31:0] i_write_data;
    reg [2:0] i_amo_op;

    // Outputs
    wire [31:0] o_read_data1;
    wire [31:0] o_read_data2;

    // Instantiate the Register File
    reg_file uut (
        .clk(clk),
        .rst_n(rst_n),
        .i_re(i_re),
        .i_wr(i_wr),
        .i_amo_en(i_amo_en),
        .i_rs1(i_rs1),
        .i_rs2(i_rs2),
        .i_rd(i_rd),
        .i_write_data(i_write_data),
        .i_amo_op(i_amo_op),
        .o_read_data1(o_read_data1),
        .o_read_data2(o_read_data2)
    );

    // Clock Generation
    always #5 clk = ~clk;

    // Test Procedure
    initial begin
        // Initialize signals
        clk = 0;
        rst_n = 0;
        i_re = 0;
```

```
        i_wr = 0;
        i_amo_en = 0;
        i_rs1 = 0;
        i_rs2 = 0;
        i_rd = 0;
        i_write_data = 0;
        i_amo_op = 0;

        // Reset the register file
        #10 rst_n = 1;
        $display("==== Register File Test Started ====");

        // Test 1: Write to Register 5
        #10 i_wr = 1; i_rd = 5; i_write_data = 32'hA;
        #10 i_wr = 0;

        // Test 2: Read from Register 5
        #10 i_re = 1; i_rs1 = 5;
        #10;
        if (o_read_data1 !== 32'hA)
            $display("ERROR: Register 5 read failed!");

        // Test 3: Perform AMOADD on Register 5 (A + 5 = F)
        #10 i_amo_en = 1; i_rs1 = 5; i_rd = 5; i_write_data = 32'h5; i_amo_op =
3'b000;
        #10 i_amo_en = 0;

        // Verify AMOADD result
        #10 i_rs1 = 5; i_re = 1;
        #10;
        if (o_read_data1 !== 32'hF)
            $display("ERROR: AMOADD failed!");

        // Test 4: Perform AMOXOR on Register 5 (F XOR 3 = C)
        #10 i_amo_en = 1; i_rs1 = 5; i_rd = 5; i_write_data = 32'h3; i_amo_op =
3'b001;
        #10 i_amo_en = 0;

        // Verify AMOXOR result
        #10 i_rs1 = 5; i_re = 1;
        #10;
        if (o_read_data1 !== 32'hC)
            $display("ERROR: AMOXOR failed!");

        // Test 5: Perform AMOAND on Register 5 (C AND 7 = 4)
        #10 i_amo_en = 1; i_rs1 = 5; i_rd = 5; i_write_data = 32'h7; i_amo_op =
```

```
3'b010;
        #10 i_amo_en = 0;

        // Verify AMOAND result
        #10 i_rs1 = 5; i_re = 1;
        #10;
        if (o_read_data1 !== 32'h4)
            $display("ERROR: AMOAND failed!");

        // Test 6: Perform AMOOR on Register 5 (4 OR 8 = C)
        #10 i_amo_en = 1; i_rs1 = 5; i_rd = 5; i_write_data = 32'h8; i_amo_op =
3'b011;
        #10 i_amo_en = 0;

        // Verify AMOOR result
        #10 i_rs1 = 5; i_re = 1;
        #10;
        if (o_read_data1 !== 32'hC)
            $display("ERROR: AMOOR failed!");

        // Test 7: Perform AMOSWAP on Register 5 (Set to 1A)
        #10 i_amo_en = 1; i_rs1 = 5; i_rd = 5; i_write_data = 32'h1A; i_amo_op =
3'b100;
        #10 i_amo_en = 0;

        // Verify AMOSWAP result
        #10 i_rs1 = 5; i_re = 1;
        #10;
        if (o_read_data1 !== 32'h1A)
            $display("ERROR: AMOSWAP failed!");

        // End simulation
        $display("==== Register File Test Completed ====");
        $stop;
    end

endmodule
```